

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Organización de Lenguajes y Compiladores 1 "A"
Ing. Mario Jose Bautista Fuentes
Aux.Carlos Fernando Enrique López García



Manual tecnico Fase 1 Proyecto

Estudiante	Carné
Ronald Renán Samayoa Martínez	202031046

Guatemala 12 de junio de 2026

INTRODUCCIÓN

El siguiente manual se ha desarrollado con la finalidad de dar a conocer la información necesaria para realizar mantenimiento, ejecución y exploración del código de la primera fase del proyecto, la cual consta del desarrollo de una aplicación de intérprete para el lenguaje GoLite, utilizando la interfaz gráfica Java Swing. Todo ha sido trabajado desde el IDE Visual Studio Code.

El manual ofrece la información necesaria de ¿cómo está realizado el código? para que una persona (Desarrollador en el lenguaje Java) que quiera editar el código lo haga de una manera apropiada, dando a conocer la estructura del desarrollo del aplicativo.

El manual técnico hace referencia a la información necesaria con el fin de orientar al personal en la concepción, planteamiento, análisis, programación e instalación del sistema.

Es de notar que la redacción propia del manual técnico está orientada a personal con conocimientos en sistemas y tecnologías de informática y conocimientos de programación sobre el lenguaje Java, además de hacer uso de las herramientas JFLEX y CUP .



DESARROLLO

Para el desarrollo del código se utilizó un IDE, el cual se trata de Visual Studio Code version 1.98.0. Utilizando el lenguaje Java y sus herramientas JFLEX y CUP. Además de usar Apache Maven en su version 3.8.7.

Prerrequisitos

- Mínimo 4 GB de memoria RAM
- Sistema operativo, preferentemente Linux o Windows
- Se necesita la instalación de una versión de Java. Una de las consideraciones importantes durante la instalación es el orden en el que se instalan los componentes de Java, específicamente el JDK (Java Development Kit) y el JRE (Java Runtime Environment), además de tener una versión de maven instalada.



ALGORITMOS DE MODELAMIENTO

Para el desarrollo del código del proyecto, se ha seguido una arquitectura modular orientada a responsabilidades específicas. Cada componente del sistema cumple una función determinada dentro del proceso de análisis de código fuente GoLite, permitiendo mantener una separación clara entre la interfaz gráfica, el procesamiento de archivos, el análisis léxico, el análisis sintáctico y la generación de reportes.

La aplicación fue implementada utilizando el lenguaje Java, apoyándose en las herramientas JFlex para la generación automática del analizador léxico y CUP para la construcción del analizador sintáctico.

A nivel general, el sistema se encuentra dividido en los siguientes módulos:

- Interfaz Gráfica de Usuario (GUI).
- Gestión de archivos.
- Analizador Léxico.
- Analizador Sintáctico.
- Manejo de errores.
- Generación de reportes.

El flujo de ejecución inicia cuando el usuario ingresa o carga código fuente dentro del editor integrado. Posteriormente, el contenido es enviado al analizador léxico para obtener la secuencia de tokens. Dichos tokens son procesados por el analizador sintáctico, el cual valida que la entrada cumpla con la gramática definida para el lenguaje GoLite. Finalmente, los resultados son mostrados mediante la consola y los reportes disponibles dentro de la aplicación.

Paquete principal (golite)

Clase Main

Constituye el punto de entrada de la aplicación. Su única responsabilidad es iniciar la interfaz gráfica principal del sistema.

```
public class Main {  
    Run main | Debug main | Run | Debug  
    public static void main(String[] args) {  
        SwingUtilities.invokeLater(() -> {  
            MainFrame ventana = new MainFrame();  
            ventana.setVisible(b: true);  
        });  
    }  
}
```

Paquete GUI

Clase Mainframe

representa la ventana principal del sistema. Actúa como controlador general de la aplicación,

coordinando la interacción entre la interfaz gráfica, el editor de código, la consola, los analizadores y los reportes. Durante su construcción se ejecuta el método `initComponents()`, encargado de crear todos los elementos visuales de la aplicación.

Además de la gestión visual, esta clase implementa la lógica relacionada con: Creación de nuevos archivos, Apertura de archivos existentes, Guardado de archivos, Cierre de pestañas, Ejecución del análisis léxico, Ejecución del análisis sintáctico, y Visualización de reportes.

```
public class MainFrame extends JFrame {  
    private List<Token> ultimoReporteTokens;  
    private List<ErrorCompilador> ultimoReporteErrores;  
    private JTabbedPane tabbedEditor;  
    private ConsolePanel consola;  
    private int contadorArchivos = 1;  
    private JLabel lblEstado;  
  
    public MainFrame() {  
        initComponents();  
    }  
}
```

```
private EditorPanel getEditorActual(){  
    return (EditorPanel) tabbedEditor.getSelectedComponent();  
}  
  
private void guardarEditor(EditorPanel editor, String nombreArchivo){  
    try {  
        FileWriter writer = new FileWriter(nombreArchivo);  
  
        writer.write(editor.getTexto());  
        writer.close();  
        editor.setModificado(modificado: false);  
    } catch (Exception ex) {  
        consola.append("Error al guardar: " + ex.getMessage());  
    }  
}  
  
private void actualizarEstado(){  
    if (lblEstado == null){  
        return;  
    }  
  
    EditorPanel editor = getEditorActual();  
  
    if (editor!=null){  
        lblEstado.setText("Linea: " + editor.getLineaActual()  
            + " Columna: " + editor.getColumnaActual());  
    }  
}
```

```

private boolean hayCambiosSinGuardar(){
    for(int i = 0; i < tabbedEditor.getTabCount(); i++){
        EditorPanel editor = (EditorPanel) tabbedEditor.getComponentAt(i);
        if(editor.isModificado()){
            return true;
        }
    }
    return false;
}

```

Clase EditorPanel

Implementa el área de edición de código fuente utilizada por el usuario para escribir programas en GoLite. Hereda de JScrollPane y encapsula un componente JTextArea, permitiendo la edición de texto con desplazamiento vertical y horizontal.

Adicionalmente incorpora un mecanismo de detección de modificaciones mediante un DocumentListener. Cada vez que el contenido cambia, se marca el documento como modificado para evitar pérdidas accidentales de información.

```

public class EditorPanel extends JScrollPane{
    private JTextArea txtEditor;
    private boolean modificado = false;

    public EditorPanel() {
        txtEditor = new JTextArea();
        txtEditor.getDocument().addDocumentListener(
            new javax.swing.event.DocumentListener() {

                @Override
                public void insertUpdate(javax.swing.event.DocumentEvent e) {
                    modificado = true;
                }

                @Override
                public void removeUpdate(javax.swing.event.DocumentEvent e) {
                    modificado = true;
                }

                @Override
                public void changedUpdate(javax.swing.event.DocumentEvent e) {
                    modificado = true;
                }
            }
        );
        setViewportView(txtEditor);
    }

    public String getTexto(){
        return txtEditor.getText();
    }

    public void setTexto(String texto){
        txtEditor.setText(texto);
    }
}

```

```

public void setTexto(String texto){
    txtEditor.setText(texto);
}

public JTextArea gTextArea(){
    return txtEditor;
}

public boolean isModificado(){
    return modificado;
}

public void setModificado(boolean modificado){
    this.modificado = modificado;
}

public int getLineaActual(){
    try {
        int pos = txtEditor.getCaretPosition();
        return txtEditor.getLineOfOffset(pos)+1;
    } catch (Exception e) {
        return 1;
    }
}

public int getColumnaActual(){
    try {
        int pos = txtEditor.getCaretPosition();
        int linea = txtEditor.getLineOfOffset(pos);
        return pos - txtEditor.getLineStartOffset(linea)+1;
    } catch (Exception e) {
        return 1;
    }
}

```

Clase ConsolePanel

Implementa la consola de salida utilizada para mostrar información generada por los analizadores.

La consola se construye mediante un JTextArea de solo lectura contenido dentro de un JScrollPane. Su propósito es presentar:

- Tokens reconocidos.
- Resultados del análisis sintáctico.
- Mensajes informativos.
- Errores producidos durante la ejecución.

```
public class ConsolePanel extends JScrollPane{
    private JTextArea txtConsola;

    public ConsolePanel(){
        txtConsola = new JTextArea();
        txtConsola.setEditable(b: false);
        setViewportView(txtConsola);
    }

    public void append(String texto){
        txtConsola.append(texto+ "\n");
    }

    public void clear(){
        txtConsola.setText(t: "");
    }
}
```

Clase ArchivoTab

Fue diseñada para representar la información asociada a una pestaña de edición dentro del sistema. Su propósito principal es almacenar información relacionada con el archivo físico vinculado a una pestaña y el estado de modificación de dicho contenido.

Esta clase funciona como una estructura de datos simple encargada de encapsular la información necesaria para administrar archivos abiertos dentro del editor.

Cada instancia puede asociarse a un archivo del sistema operativo y mantener un indicador que permita determinar si existen cambios pendientes de guardar.

```
public class ArchivoTab {
    private File archivo;
    private boolean modificado;

    public ArchivoTab(){
        this.archivo = null;
        this.modificado = false;
    }

    public File getArchivo(){
        return archivo;
    }

    public void setArchivo(File archivo){
        this.archivo = archivo;
    }

    public boolean isModificado(){
        return modificado;
    }

    public void setModificado(boolean modificado){
        this.modificado = modificado;
    }
}
```

Paquete Lexer

Enum TipoToken

Define todas las categorías léxicas reconocidas por el lenguaje GoLite implementado en el proyecto.

Durante el análisis léxico, cada lexema identificado es clasificado dentro de una categoría específica representada por un valor de esta enumeración. De esta manera, el analizador sintáctico puede trabajar con tipos de tokens estandarizados en lugar de procesar cadenas de texto directamente.

Clase Token

Representa la unidad mínima de información producida por el análisis léxico. Cada vez que el analizador léxico reconoce un lexema válido, se crea un objeto Token que almacena toda la información necesaria para identificarlo y ubicarlo dentro del código fuente.

La información almacenada incluye:

- Tipo de token.
- Lexema reconocido.

- Línea de aparición.
- Columna de aparición

```
public class Token {  
    private TipoToken tipo;  
    private String lexema;  
    private int linea ;  
    private int columna;  
  
    public Token(TipoToken tipo, String lexema, int linea, int columna){  
        this.tipo = tipo;  
        this.lexema = lexema;  
        this.linea = linea;  
        this.columna = columna;  
    }  
  
    public TipoToken getTipo(){  
        return tipo;  
    }  
  
    public String getLexema(){  
        return lexema;  
    }  
  
    public int getLinea(){  
        return linea;  
    }  
  
    public int getColumna(){  
        return columna;  
    }  
}
```

Clase AnalizadorLexico

Actúa como una capa de abstracción entre la interfaz gráfica y el analizador léxico generado automáticamente mediante JFlex.

Su función principal consiste en recibir el código fuente ingresado por el usuario, ejecutar el proceso de análisis léxico y devolver la lista de tokens reconocidos.

Cuando el usuario ejecuta el análisis desde la interfaz gráfica, el código fuente es enviado al método analizar().

Internamente, el texto es convertido en un flujo de caracteres mediante un objeto `StringReader`, el cual es utilizado para construir una instancia del analizador generado por JFlex (Lexer).

Posteriormente, se realiza un recorrido completo de la entrada utilizando llamadas sucesivas al método `yylex()`, obteniendo un token en cada iteración hasta alcanzar el token especial EOF.

```
public class AnalizadorLexico {
    private List<ErrorCompilador> errores = new ArrayList<>();

    public List<ErrorCompilador> getErrores(){
        return errores;
    }

    public List<Token> analizar(String entrada){
        List<Token> tokens = new ArrayList<>();

        try {
            Lexer lexer = new Lexer(new StringReader(entrada));

            Token token;

            do {
                token = lexer.yylex();
                if (token != null){
                    tokens.add(token);

                    if(token.getTipo() == TipoToken.ERROR){
                        errores.add(new ErrorCompilador(tipo: "Lexico",
                            "Caracter no reconocido: " + token.getLexema(),
                            token.getLinea(), token.getColumna()));
                    }
                }
            } while (token != null && token.getTipo() != TipoToken.EOF);

        } catch (Exception e) {
            e.printStackTrace();
        }

        return tokens;
    }
}
```

Paquete Errors

Clase ErrorCompilador

Permite representar errores detectados durante las distintas fases del procesamiento del código fuente.

La clase encapsula toda la información necesaria para describir un error de manera precisa y comprensible para el usuario.

Su diseño permite registrar errores provenientes de diferentes etapas del sistema, tales como:

- Errores léxicos.
- Errores sintácticos.
- Posibles errores semánticos en futuras ampliaciones.

```
public class ErrorCompilador {
    private String tipo;
    private String descripcion;
    private int linea;
    private int columna;

    public ErrorCompilador(String tipo, String descripcion, int linea, int columna){
        this.tipo=tipo;
        this.descripcion=descripcion;
        this.linea=linea;
        this.columna=columna;
    }

    public String getTipo(){
        return tipo;
    }

    public String getDescripcion(){
        return descripcion;
    }

    public int getLinea(){
        return linea;
    }

    public int getColumna(){
        return columna;
    }
}
```

Paquete Parser

Clase AnalizadorSintactico

Funciona como intermediario entre la aplicación y el parser generado mediante CUP.

Su propósito es validar que la secuencia de tokens producida por el análisis léxico cumpla con las reglas gramaticales definidas para el lenguaje GoLite.

Funcionamiento

Al recibir el código fuente, la clase crea una nueva instancia del analizador léxico compatible con CUP (LexerCup), el cual se encarga de suministrar los tokens requeridos por el parser.

Posteriormente se construye una instancia de la clase parser generada automáticamente por CUP y se inicia el proceso de análisis mediante el método parse().

```

public class AnalizadorSintactico {
    private List<ErrorCompilador> errores = new ArrayList<>();

    public List<ErrorCompilador> getErrores(){
        return errores;
    }

    public String analizar(String entrada){
        errores.clear();

        try{
            LexerCup lexer = new LexerCup(new StringReader(entrada));
            parser sintactico = new parser(lexer);
            sintactico.parse();

            errores.addAll(sintactico.getErrores());

            if(errores.isEmpty()){
                return "Análisis sintactico correcto";
            }

            return "Análisis sintactico con errores";

        }catch(Exception e){
            errores.add(
                new ErrorCompilador(
                    tipo: "Sintactico",
                    e.getMessage(),
                    linea: 0,
                    columna: 0
                )
            );
            return "Error sintactico";
        }
    }
}

```

Paquete resources

Subpaquete flex

Lexer.flex (Analizador lexico generado con JFLEX)

Contiene la especificación léxica del lenguaje GoLite y es utilizado por JFlex para generar automáticamente la clase Lexer.

Su función principal consiste en reconocer los diferentes componentes léxicos presentes en el código fuente y transformarlos en objetos Token que posteriormente serán utilizados para reportes y visualización de resultados

```
%%

%public
%class Lexer
%unicode
%line
%column
%type Token

%{
    private Token token (TipoToken tipo, String lexema){
        return new Token(tipo, lexema, yyline + 1, yycolumn + 1);
    }
%}
```

LexerCup.flex (Analizador lexico para CUP)

Contiene una segunda especificación léxica diseñada específicamente para integrarse con el parser generado por CUP.

A diferencia del Lexer principal, este analizador no genera objetos Token, sino objetos Symbol compatibles con la infraestructura interna de CUP.

El comportamiento general es similar al analizador léxico principal, ya que utiliza las mismas expresiones regulares para reconocer los elementos del lenguaje.

```
%%

%public
%class LexerCup
%unicode
%cup
%line
%column

%{
    private Symbol symbol (int type){
        return new Symbol(type, yyline + 1, yycolumn + 1);
    }

    private Symbol symbol (int type, Object value){
        return new Symbol(type, yyline + 1, yycolumn + 1, value);
    }
%}
```

Parser.cup

Contiene la definición formal de la gramática del lenguaje GoLite y es utilizado para generar automáticamente el parser encargado de validar la estructura sintáctica de los programas.

La especificación define:

- Terminales.
- No terminales.
- Reglas de producción.
- Precedencias.
- Asociatividades.
- Mecanismos de recuperación de errores.

```
start with INICIO;

INICIO ::=
| LISTA_SENTENCIAS
;

LISTA_SENTENCIAS ::=
| LISTA_SENTENCIAS SENTENCIA | SENTENCIA
;

SENTENCIA ::=
| DECLARACION | ASIGNACION | PRINT | IF_SENTENCIA | FOR_SENTENCIA
| BREAK_SENTENCIA | CONTINUE_SENTENCIA | BLOQUE
| error PUNTO_COMA
| {
|   System.out.println("Sentencia invalida recuperada");
| }
;

;
```

